

CHATBOTS COMERCIALES

Guia tecnica para crear, integrar y replicar agentes comerciales con Google ADK

ERP omnicanal, CRM y agente comercial

Version 1.0 | 2026-08-08

Guia de tecnologias, desarrollo, configuracion y operacion

Version: 1.0

Fecha: agosto de 2026

Producto: ERP omnicanal, CRM y agente comercial

Framework de agentes: Google ADK

Documento: Guia tecnica para desarrolladores y replicacion por cliente

1. Objetivo

Esta guia define como crear, configurar, probar, desplegar y replicar un chatbot comercial conectado al ERP.

El bot debe poder:

- Recibir y enviar mensajes por WhatsApp.
- Crear y actualizar contactos sin duplicados.
- Consultar productos y servicios.
- Responder preguntas frecuentes y conocimiento empresarial.
- Calificar prospectos.
- Mover oportunidades por un embudo comercial.
- Consultar disponibilidad y agendar citas.
- Crear, revisar y enviar cotizaciones.
- Escalar conversaciones a un asesor humano.
- Registrar eventos, costos, errores y auditoria.
- Trabajar con Gmail, Google Sheets, Google Drive y Google Calendar.

El agente no debe tener acceso libre a toda la base de datos. Solo puede solicitar tools autorizadas y cada tool valida empresa, usuario, permisos, datos y confirmaciones.

2. Arquitectura de referencia

```
Cliente
|
+-- WhatsApp / WATI / Meta Cloud API / WAHA
|
v
Webhook HTTPS
|
v
ERP API (Node.js + Express)
|
+-- PostgreSQL: fuente transaccional
+-- Redis: cache, locks y cola
+-- Google APIs: Gmail, Sheets, Drive, Calendar
+-- WATI o Meta: mensajeria y bandeja humana
|
v
Agent Service (Python + Google ADK)
|
+-- Tools de CRM
+-- Tools de productos
```

```
+-- Tools de agenda
+-- Tools de cotizacion
+-- Tools de conocimiento
+-- Tool de handoff
|
v
Modelo LLM (Groq mediante LiteLLM, Gemini u otro proveedor compatible)
```

Principio de separacion

- El ERP es el sistema de registro y permisos.
- PostgreSQL es la fuente de verdad.
- Google ADK es el sistema de inteligencia y orquestacion.
- WATI o Meta Cloud API es el sistema de mensajeria.
- Google Calendar es la fuente de disponibilidad.
- Google Sheets es una interfaz de carga, configuracion o exportacion, no la base transaccional final.

3. Catalogo de tecnologias

Categoria	Tecnologia	Funcion
Lenguaje ERP	Node.js 20+	API y aplicacion comercial
Framework ERP	Express 5	Rutas HTTP y middleware
Frontend actual	HTML, CSS y Vanilla JS	Interfaz existente del ERP
Frontend objetivo	Next.js, React y Tailwind CSS	Migracion futura de la interfaz
Lenguaje agente	Python 3.11+	Runtime del agente
Framework agente	Google ADK	Agentes, sesiones, tools y workflows
Interfaz de desarrollo	adk web	Playground visual para probar el agente
Adaptador LLM	LiteLLM	Cambiar de proveedor sin rehacer tools
Modelo economico	Groq GPT-OSS 20B	Conversaciones normales
Modelo avanzado	Groq GPT-OSS 120B	Casos complejos
Base de datos	PostgreSQL	Datos comerciales y auditoria
ORM/driver	SQLAlchemy/Psycopg o pg	Acceso a PostgreSQL
Cache y cola	Redis + BullMQ/Celery	Reintentos, locks y tareas largas
Mensajeria	WATI o WhatsApp Cloud API	Enviar y recibir WhatsApp
Alternativa	WAHA	Pilotos o clientes especificos
Email	Gmail API	Mensajes y notificaciones
Hojas	Google Sheets API	Catalogos, cargas y exportaciones
Archivos	Google Drive o S3 compatible	Documentos, imagenes y PDFs
Agenda	Google Calendar API	Disponibilidad y citas
Comercio	Shopify GraphQL	Productos e inventario opcional
Pagos	Proveedor con webhooks	Links, cobros y conciliacion
Documentos	PDF service	Cotizaciones y contratos
Eventos	Event log propio	Analitica y auditoria
Observabilidad	OpenTelemetry y Sentry	Trazas, errores y costos
Contenedores	Docker	Empaquetado reproducible

Categoria	Tecnologia	Funcion
Despliegue	EasyPanel	Servidor por cliente
CI/CD	GitHub Actions	Pruebas y despliegue
Testing	pytest, Vitest/Supertest	Pruebas de agente y API

4. Modelo por cliente

Cada cliente debe tener una instalacion aislada:

```

Cliente A
+-- Proyecto Google Cloud A
+-- PostgreSQL A
+-- WATI o Meta A
+-- EasyPanel A
+-- Dominio A
+-- Credenciales A

```

```

Cliente B
+-- Proyecto Google Cloud B
+-- PostgreSQL B
+-- WATI o Meta B
+-- EasyPanel B
+-- Dominio B
+-- Credenciales B

```

Aunque cada cliente tenga servidor propio, todos los modelos deben incluir `workspace_id`. Esto permite evolucionar a una plataforma multiempresa sin rehacer el dominio.

Entidades principales

```

workspaces
users
roles
permissions
integrations
contacts
conversations
messages
pipelines
pipeline_stages
products
quotes
quote_versions
quote_items
payment_plans
contracts
accounts_receivable
payments
appointments
automations
domain_events
audit_events

```

5. Regla de autorizacion

Antes de cualquier lectura o escritura:

```

SI el usuario pertenece al workspace
Y su rol permite la accion
Y el registro pertenece al workspace
Y la integracion esta habilitada
ENTONCES ejecutar
DE LO CONTRARIO rechazar y auditar

```

Roles base

Rol	Acciones
Administrador	Configuracion, usuarios, integraciones y datos
Supervisor	Equipo, contactos, conversaciones y reportes
Asesor	Conversaciones y oportunidades asignadas
Consulta	Solo lectura autorizada
Agente IA	Solo tools declaradas y permitidas

Nunca se debe considerar identidad valida un header enviado por el navegador como `x-user-id` o `x-user-name`.

6. Instalacion local

ERP Node.js

```
git clone https://github.com/AXELONGO/APP-ERP-COMERCIAL.git
cd APP-ERP-COMERCIAL
npm ci
cp .env.example .env
npm start
```

PostgreSQL local

```
docker run --name erp-postgres \
-e POSTGRES_USER=erp_user \
-e POSTGRES_PASSWORD=change_me \
-e POSTGRES_DB=erp_comercial \
-p 5432:5432 \
-d postgres:16-alpine
```

Agente ADK

```
python3.11 -m venv .venv
source .venv/bin/activate
pip install google-adk litellm fastapi uvicorn pydantic-settings httpx
export GOOGLE_GENAI_USE_VERTEXAI=FALSE
adk web
```

La interfaz de `adk web` se utiliza para seleccionar el agente, enviar mensajes, observar eventos, revisar llamadas de tools y detectar errores. No debe exponerse publicamente sin autenticacion.

7. Configuracion de variables

ERP

```
NODE_ENV=production
PORT=3000
PUBLIC_BASE_URL=https://erp-cliente.example.com
DATABASE_URL=postgresql://erp_user:password@postgres:5432/erp_comercial
REDIS_URL=redis://redis:6379
ERP_AUTH_SECRET=generar_un_secreto_largo
```

IA

```
GROQ_API_KEY=clave_del_cliente
GROQ_MODEL=openai/gpt-oss-20b
GROQ_MAX_OUTPUT_TOKENS=800
AI_MAX_TOOL_CALLS=5
AI_MAX_COST_PER_CONVERSATION=0.02
```

WATI

```
WATI_API_URL=https://api.wati.io
WATI_API_TOKEN=token_del_workspace
WATI_WEBHOOK_SECRET=secreto_de_firma
```

Google

```
GOOGLE_CLIENT_ID=cliente_oauth
GOOGLE_CLIENT_SECRET=secreto_oauth
GOOGLE_REDIRECT_URI=https://erp-cliente.example.com/api/auth/google/callback
GOOGLE_SHEETS_ID=id_de_hoja
GOOGLE_DRIVE_ROOT_FOLDER_ID=id_de_carpeta
GOOGLE_CALENDAR_ID=primary
GMAIL_FROM_ADDRESS=ventas@cliente.com
```

Nunca subir `.env`, `credentials.json`, refresh tokens, API keys o service accounts a GitHub.

8. Configuración de Google Cloud

Crear un proyecto por cliente y activar:

- Gmail API.
- Google Sheets API.
- Google Drive API.
- Google Calendar API.

OAuth 2.0

Usar OAuth cuando el bot deba actuar con la cuenta de un usuario o de un equipo.

Scopes sugeridos:

```
https://www.googleapis.com/auth/gmail.send
https://www.googleapis.com/auth/spreadsheets
https://www.googleapis.com/auth/drive.file
https://www.googleapis.com/auth/calendar
```

Solicitar solo los permisos necesarios. Guardar refresh tokens cifrados y asociados al cliente y usuario.

Service Account

Puede utilizarse para Sheets y Drive cuando el cliente comparte una hoja o carpeta con el email de la cuenta de servicio. Para Gmail y Calendar de usuarios individuales se recomienda OAuth, salvo que Google Workspace permita delegación de dominio.

9. Integración de WhatsApp

WATI

WATI puede funcionar como capa de WhatsApp, contactos, conversaciones, inbox humano, plantillas y campañas.

Flujo:

```
WhatsApp
-> WATI
-> webhook message.received
-> ERP
-> Google ADK
-> WATI API
-> respuesta al cliente
```

Meta WhatsApp Cloud API

Es la alternativa oficial directa. Requiere Meta Business, numero conectado, webhooks y plantillas aprobadas.

Interfaz comun

El agente no debe conocer el proveedor. Crear un adaptador:

```
MessagingProvider
- receive_message()
- send_message()
- send_template()
- send_media()
- get_conversation()
- handoff_to_human()
```

Fuera de la ventana de atencion de WhatsApp se deben usar plantillas aprobadas y respetar consentimiento y opt-out.

10. Flujo general del mensaje

1. Recibir webhook HTTPS
2. Validar firma HMAC
3. Deduplicar por message_id
4. Identificar workspace y canal
5. Normalizar telefono a E.164
6. Buscar o crear contacto
7. Buscar conversacion abierta
8. Crear conversacion si no existe
9. Guardar mensaje entrante
10. Verificar si IA esta activa
11. Ejecutar agente y tools autorizadas
12. Enviar respuesta o crear handoff
13. Guardar mensaje saliente
14. Actualizar embudo y metricas
15. Registrar evento de auditoria

El webhook debe responder rapidamente con 2xx. El procesamiento largo se ejecuta en una cola.

11. Tools obligatorias

Agenda

Tool	Funcion
scheduling_list_appointment_types	Lista servicios, duraciones y calendarios disponibles
scheduling_check_availability	Calcula horarios libres considerando calendario y reglas
scheduling_book	Crea una cita despues de confirmar datos y disponibilidad
scheduling_reschedule	Cambia la fecha u hora y vuelve a validar disponibilidad
scheduling_cancel	Cancela una cita y registra motivo
scheduling_update	Actualiza notas, contacto, asesor o servicio sin cambiar horario
scheduling_list_appointments	Lista citas por contacto, asesor, estado o periodo
scheduling_lookup_appointment_by_phone	Busca citas por telefono normalizado

Las operaciones de reservar, reprogramar y cancelar requieren confirmacion explicita.

Conocimiento

Tool	Funcion
context_search_knowledge	Busca en documentos, políticas, manuales y conocimiento empresarial
search_faq	Responde preguntas frecuentes estructuradas

context_search_knowledge sirve para búsquedas amplias. search_faq sirve para preguntas puntuales.

Productos

Tool	Funcion
context_search_products	Busca productos por intencion y lenguaje natural
get_product	Obtiene detalle de un producto por ID o SKU
list_products	Lista productos con filtros, orden y paginacion

Contacto y humano

Tool	Funcion
confirm_contact_name	Confirma identidad antes de acciones relevantes
handoff_to_human	Pausa la IA, asigna asesor y registra motivo y resumen

12. Contrato de una tool

Cada tool debe tener:

- Nombre estable.
- Descripcion corta.
- Parametros tipados.
- Campos requeridos.
- Validacion de workspace.
- Validacion de permisos.
- Confirmacion si es destructiva.
- Resultado estructurado.
- Errores clasificados.
- Idempotencia cuando modifique datos.
- Registro de auditoria.

Ejemplo conceptual:

```
from pydantic import BaseModel

class AvailabilityRequest(BaseModel):
    appointment_type_id: str
    date_from: str
    date_to: str
    timezone: str
    advisor_id: str | None = None

def scheduling_check_availability(request: AvailabilityRequest):
    # Validar workspace y permisos.
    # Consultar Google Calendar FreeBusy.
    # Aplicar horarios, descansos y duracion.
    # Devolver slots en la zona horaria solicitada.
    return {"slots": []}
```


No agregar valores default innecesarios en schemas enviados a proveedores que no los soporten.

13. Logica del agente

El agente debe identificar:

- Intencion.
- Datos disponibles.
- Datos faltantes.
- Confianza.
- Sensibilidad.
- Necesidad de humano.
- Siguiendo accion.

Regla de respuesta

```
SI el agente esta activo
Y el canal permite IA
Y ningun humano tomo la conversacion
Y la intencion no es sensible
Y existe informacion suficiente
ENTONCES responder o usar una tool de lectura
DE LO CONTRARIO pedir confirmacion o transferir a humano
```

Acciones que requieren confirmacion

- Enviar cotizacion.
- Cancelar cita.
- Cambiar precio.
- Aplicar descuento extraordinario.
- Crear contrato.
- Confirmar pago.
- Eliminar informacion.
- Crear una obligacion financiera.

La IA puede preparar un borrador, pero la aplicacion debe controlar la ejecucion final.

14. Calendario

Usar Google Calendar API con OAuth.

Reserva segura

1. Confirmar contacto
2. Seleccionar tipo de cita
3. Consultar horario laboral
4. Excluir descansos
5. Excluir eventos existentes
6. Aplicar duracion y tiempo de preparacion
7. Mostrar slots
8. Recibir confirmacion
9. Volver a verificar disponibilidad
10. Crear evento

- 11. Guardar calendar_event_id
- 12. Enviar confirmacion
- 13. Programar recordatorios

Debe guardarse la zona horaria del contacto o solicitarla cuando sea necesario.

15. Cotizaciones y pagos

Cotizacion

Una cotizacion enviada queda congelada. Los cambios crean una nueva version.

```
importe = cantidad * precio_unitario
subtotal = suma(importes)
base = subtotal - descuento
impuestos = base * tasa
total = base + impuestos
```

Guardar importes con NUMERIC, moneda y politica de impuestos.

Plan de pago

Soportar:

- Pago unico.
- Apartado y saldo.
- Anticipo y mensualidades.
- Apartado, enganche, mensualidades y pago final.
- Plan personalizado.

La suma de conceptos debe ser igual al total. Un enlace abierto no significa que el pago este confirmado; el estado pagado debe llegar por webhook firmado del proveedor.

Cobranza

```
Pendiente
Proxima a vencer
Pagada
Parcial
Vencida
Cancelada
Condonada
```

16. Automatizaciones

Cada automatizacion contiene:

```
evento
condiciones
acciones
espera_opcional
estado
historial_de_ejecuciones
```

Ejemplos:

```
contacto movido a Cotizacion
-> crear tarea
-> notificar asesor
-> preparar seguimiento

cotizacion aceptada
-> crear contrato
```

-> generar calendario de pagos

pago vencido

-> calcular mora

-> notificar asesor

-> enviar recordatorio autorizado

Procesos financieros y de permisos deben ejecutarse en el backend. n8n puede utilizarse para integraciones no críticas.

17. Auditoria y analitica

Guardar eventos historicos, no solo el estado actual:

```
contact.created  
message.received  
message.sent  
ai.responded  
human.responded  
conversation.handoff  
stage.changed  
appointment.created  
appointment.confirmed  
quote.sent  
quote.viewed  
quote.accepted  
payment.received  
payment.overdue
```

Toda operacion importante debe guardar:

```
que ocurrio  
quien lo hizo  
cuando ocurrio  
registro afectado  
valor anterior  
valor nuevo  
origen: usuario, IA, automatizacion o integracion
```

Metricas iniciales:

- Conversaciones recibidas.
 - Tiempo de primera respuesta.
 - Porcentaje respondido por IA.
 - Porcentaje escalado.
 - Conversacion a cita.
 - Cita a cotizacion.
 - Cotizacion a venta.
 - Ventas por asesor y canal.
 - Asistencia a citas.
 - Cartera vencida.
 - Tokens consumidos.
 - Costo por conversacion.
-

18. Despliegue en EasyPanel

Servicios por cliente

erp-cliente
postgres-cliente
redis-cliente
agent-cliente

WATI o Meta puede ser un servicio externo. Cada cliente debe tener dominio HTTPS propio.

Configuracion de la aplicacion

- Fuente: repositorio GitHub.
- Build: Dockerfile.
- Puerto ERP: 3000 o el definido por PORT.
- Puerto agente: 8080.
- Health check: /health.
- Variables: panel de EasyPanel.
- Logs: stdout/stderr.
- Persistencia: volumen PostgreSQL.
- Backups: programados y verificados.

No usar localhost para conectar servicios entre contenedores. Utilizar el nombre interno del servicio.

19. Seguridad de produccion

Checklist minimo:

- Autenticacion real.
 - Autorizacion por empresa y rol.
 - Secretos fuera de GitHub.
 - Refresh tokens cifrados.
 - Webhooks con HMAC.
 - Deduplicacion de eventos.
 - Rate limits por usuario, canal y proveedor.
 - CORS restringido.
 - CSP activa.
 - Archivos privados.
 - Datos sensibles fuera de logs.
 - Validacion de MIME y tamano.
 - No aceptar destinos webhook arbitrarios.
 - Backups cifrados.
 - Rotacion de claves.
 - Registro de auditoria.
 - Pruebas de permisos y aislamiento.
-

20. Pruebas

Agente

- Selección de tool correcta.
- Rechazo de parámetros inválidos.
- No ejecución de acciones sin confirmación.
- Handoff ante incertidumbre.
- Respuestas con conocimiento correcto.
- Límite de llamadas por turno.
- Límite de tokens y costo.

WhatsApp

- Firma válida e inválida.
- Eventos duplicados.
- Mensajes fuera de ventana.
- Plantillas.
- Entrega y lectura.
- Reintentos.

Agenda

- Horario ocupado.
- Reserva simultánea.
- Reprogramación.
- Cancelación.
- Zona horaria.
- Recordatorios.

Comercial

- Duplicados de contactos.
- Cotización congelada.
- Versión nueva.
- Redondeos.
- Impuestos.
- Plan que no suma 100%.
- Pago duplicado.
- Pago no confirmado.
- Mora.

Seguridad

- Acceso entre empresas.
- Permisos por rol.
- XSS.

- SSRF.
 - CORS.
 - Rate limit.
 - Fugas en logs.
 - Secretos en repositorio.
-

21. Proceso para replicar un cliente

1. Crear servidor y proyecto EasyPanel
 2. Crear proyecto Google Cloud del cliente
 3. Crear PostgreSQL y Redis
 4. Crear dominio HTTPS
 5. Configurar variables y secretos
 6. Configurar Google OAuth
 7. Conectar Gmail, Sheets, Drive y Calendar
 8. Conectar WATI o Meta Cloud API
 9. Cargar contactos y productos
 10. Configurar pipeline
 11. Configurar tipos de cita
 12. Configurar FAQs y conocimiento
 13. Configurar formas de pago
 14. Crear usuarios y roles
 15. Probar tools en adk web
 16. Probar conversación completa
 17. Probar handoff humano
 18. Probar cotización y pago
 19. Activar automatizaciones
 20. Entregar guía del cliente
 21. Activar monitoreo y backups
-

22. Roadmap recomendado

Fase 0: seguridad y base

Empresas, usuarios, roles, PostgreSQL, auditoria, secretos y webhooks.

Fase 1: CRM y conversaciones

Contactos, mensajes, conversaciones, bandeja, asignación y tiempo real.

Fase 2: embudo y productos

Etapas configurables, historial, catálogo, importación y búsqueda.

Fase 3: agenda

Google Calendar, disponibilidad, reservas, reprogramaciones y recordatorios.

Fase 4: cotizador

Cotizaciones versionadas, productos, impuestos, datos fiscales y PDFs.

Fase 5: pagos y cobranza

Planes, links, webhooks, contratos, cuentas por cobrar y mora.

Fase 6: agente ADK

Tools de lectura, confirmación, handoff, agenda, productos y cotización.

Fase 7: automatizaciones y analitica

Motor de eventos, reportes, costos, conversiones y observabilidad.

Fase 8: escala

Workers separados, Redis distribuido, colas, OpenTelemetry, CI/CD y migracion gradual del frontend a React.

23. Checklist de entrega

Cliente

- Dominio funcionando.
- Usuarios creados.
- Roles revisados.
- WhatsApp conectado.
- Gmail conectado.
- Calendar conectado.
- Drive y Sheets conectados.
- Catalogo cargado.
- FAQs cargadas.
- Pipeline configurado.
- Tipos de cita configurados.
- Formas de pago configuradas.
- Prueba de cotizacion realizada.
- Prueba de handoff realizada.
- Backup verificado.

Desarrollador

- .env no esta en Git.
 - No hay credenciales en commits.
 - Migraciones ejecutadas.
 - Health checks activos.
 - Logs sanitizados.
 - Webhooks firmados.
 - Idempotencia probada.
 - Permisos probados.
 - Costos registrados.
 - Rollback disponible.
 - adk web probado en desarrollo.
 - Documentacion del cliente entregada.
-

24. Fuentes y referencias

- Google ADK: <https://google.github.io/adk-docs/>
- Google Calendar API: <https://developers.google.com/calendar/api>
- Gmail API: <https://developers.google.com/gmail/api>
- Google Sheets API: <https://developers.google.com/sheets/api>
- Google Drive API: <https://developers.google.com/drive/api>
- WATI API: <https://docs.wati.io/>
- WATI webhooks: <https://docs.wati.io/webhooks>
- WATI calendar booking: <https://docs.wati.io/guides/calendar-booking>
- PostgreSQL: <https://www.postgresql.org/docs/>
- Docker: <https://docs.docker.com/>
- EasyPanel: <https://easypanel.io/>
- LiteLLM: <https://docs.litellm.ai/>
- Groq models: <https://console.groq.com/docs/models>

Los precios de modelos, proveedores de WhatsApp y servicios externos deben verificarse antes de cada contrato porque pueden cambiar.